Assignment 2: iPerf, TCP Statistics, and Congestion Control

CS 422: Computer Networks, Fall 2026

Group Members: Ayusha Patra, Kabileswaran Krishna Kumar, Vihaan Kodeboyina, Millan Kumar

GitHub Repo link:
https://github.com/LelsersLasers/CS422-Fall2026/tree/main/Assignment2

1a) iPerf Throughput Application

Our Python socket client communicates with standard public iPerf3 servers using separate control and data connections, without invoking the iPerf3 binary or using a custom receiver. In part1/run_test.py, lines 118–333, it opens the control connection and exchanges JSON parameters (118–146), opens the data connection (151–177), sends continuously for the configured duration (179–281), then ends the test, exchanges results, and closes both sockets (286–333).

part1/protocol.py, lines 61–224 implements cookies, exact reads, length-prefixed JSON, and protocol-state checks. Timeouts and error checks handle unresponsive/busy servers, rejected parameters, and early termination; failed tests are logged and replaced within a bounded attempt count.

1b) Destination Selection

part1/utils.py, lines 30–100 loads addresses and ports from `data/listed_iperf3_servers.json`, expands port ranges, deduplicates host/port pairs, and randomly shuffles them. `-n` sets the requested successful destination count (default 10), and `--seed` makes candidate ordering repeatable.

part1/run_iperf.py, lines 112–170 logs failures in `failures.csv` and tries replacements until the target, candidate exhaustion, or `--max-attempts` (default 30). The manifest records selected addresses/ports, attempts, and shortfall (lines 236–249). Table 1 lists ten successful destinations, so its destination shortfall is zero; the original failed-attempt count is unavailable.

1c) Goodput Measurement

At each sampling interval, we read cumulative acknowledged bytes from `TCP_INFO` on the live data socket and compute:

goodput_bps = 8 × Δbytes_acked / actual_elapsed_interval_s

This uses monotonic elapsed time, not the requested interval (part1/tcp_stats.py, lines 202–220). part1/run_iperf.py, lines 44–63 computes minimum, median, mean, and nearest-rank p95 interval goodput; raw samples and aggregate summaries are saved at lines 149–184. plot_results.py, lines 91–135 generates a goodput time-series page for every destination. Figures 1–10 and Table 1 show the ten-destination CUBIC measurements; rates are in Mbit/s.

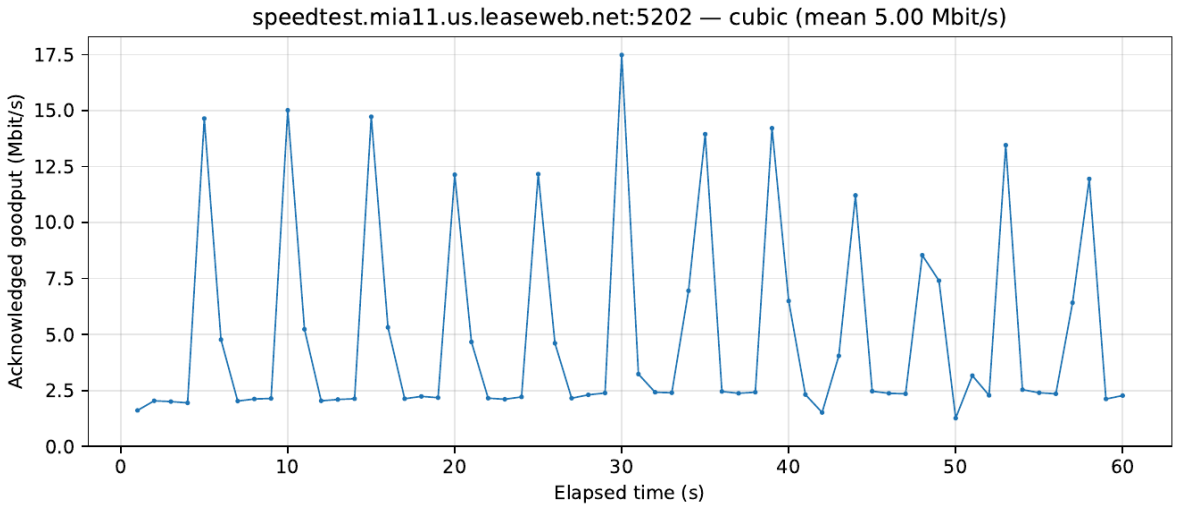


Figure 1: Goodput vs. Time - speedtest.mia11.us.leaseweb.net

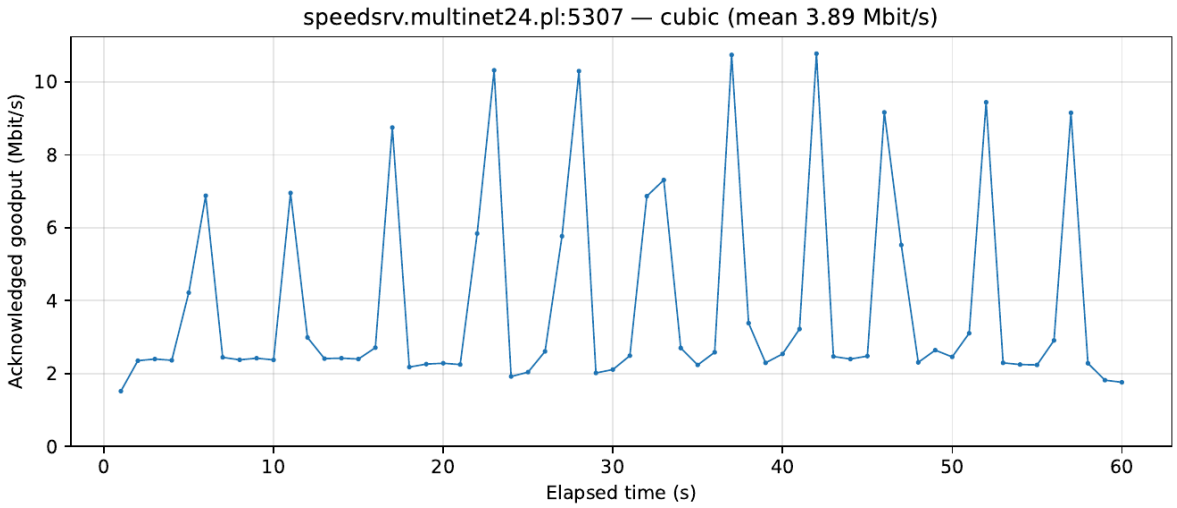


Figure 2: Goodput vs. Time - speedsrv.multinet24.pl

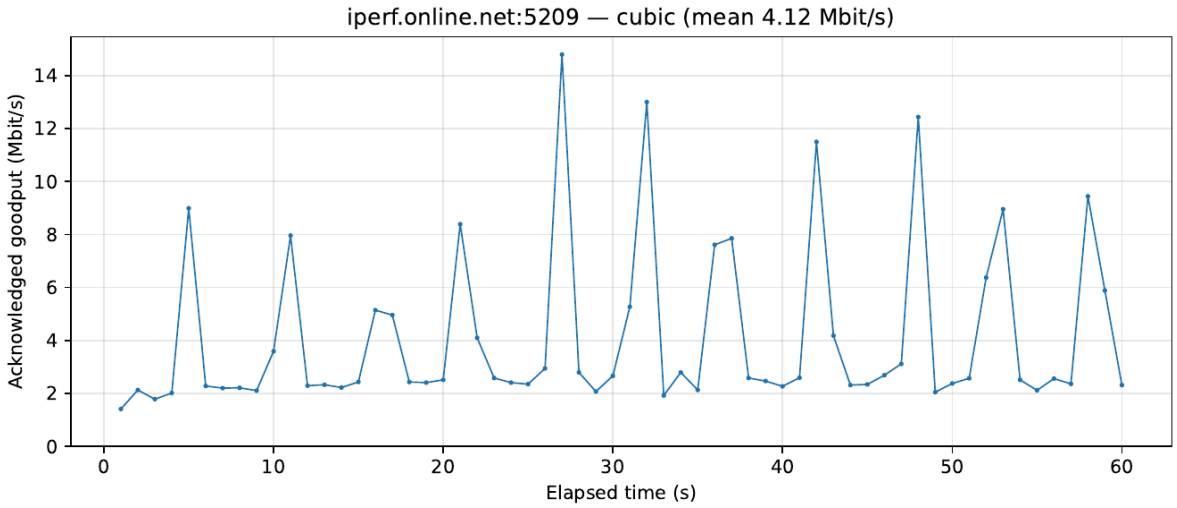


Figure 3: Goodput vs. Time - iperf.online.net

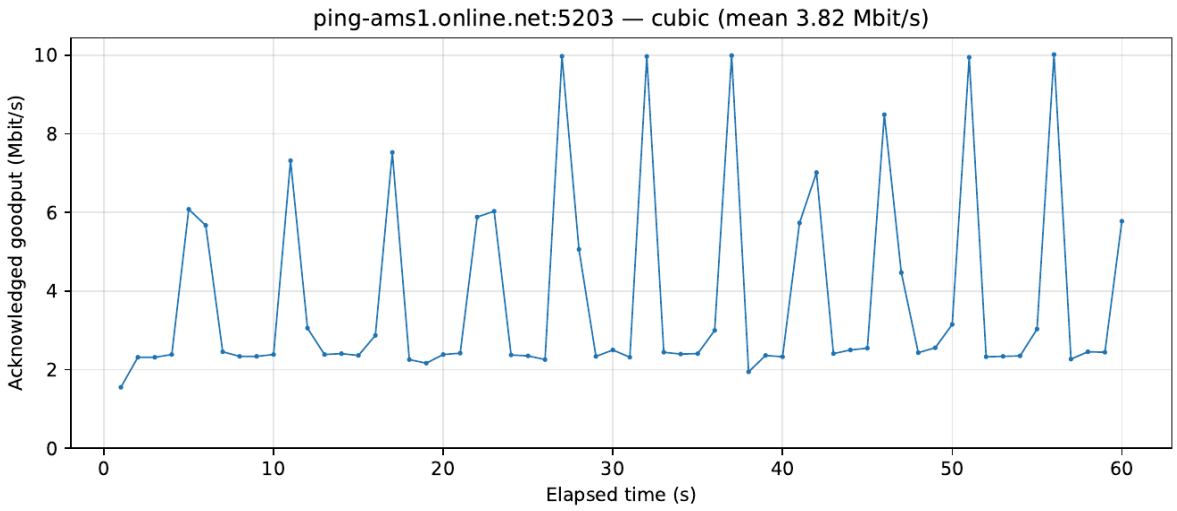


Figure 4: Goodput vs. Time - ping-ams1.online.net

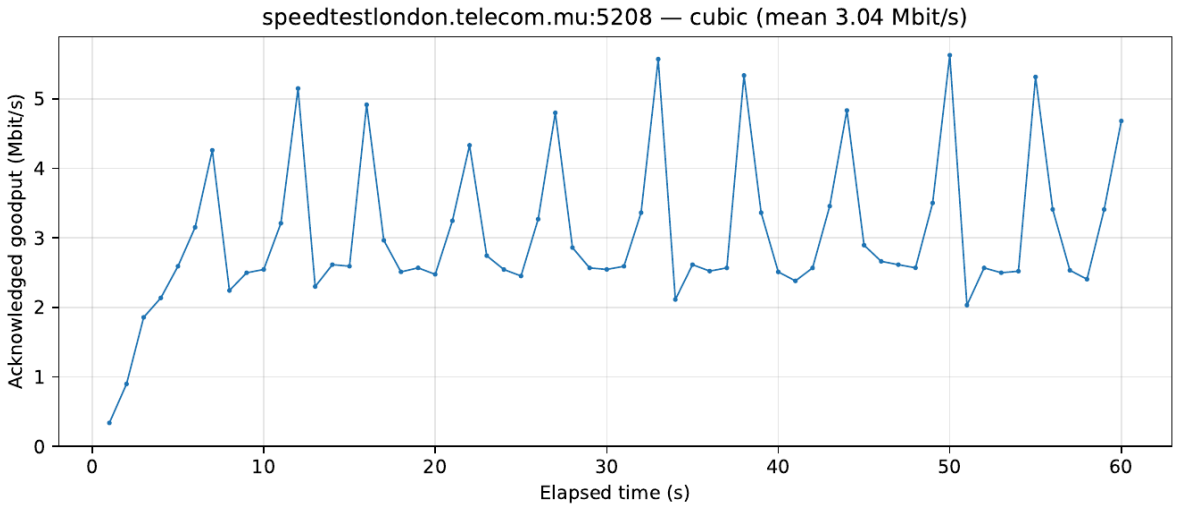


Figure 5: Goodput vs. Time - speedtestlondon.telecom.mu

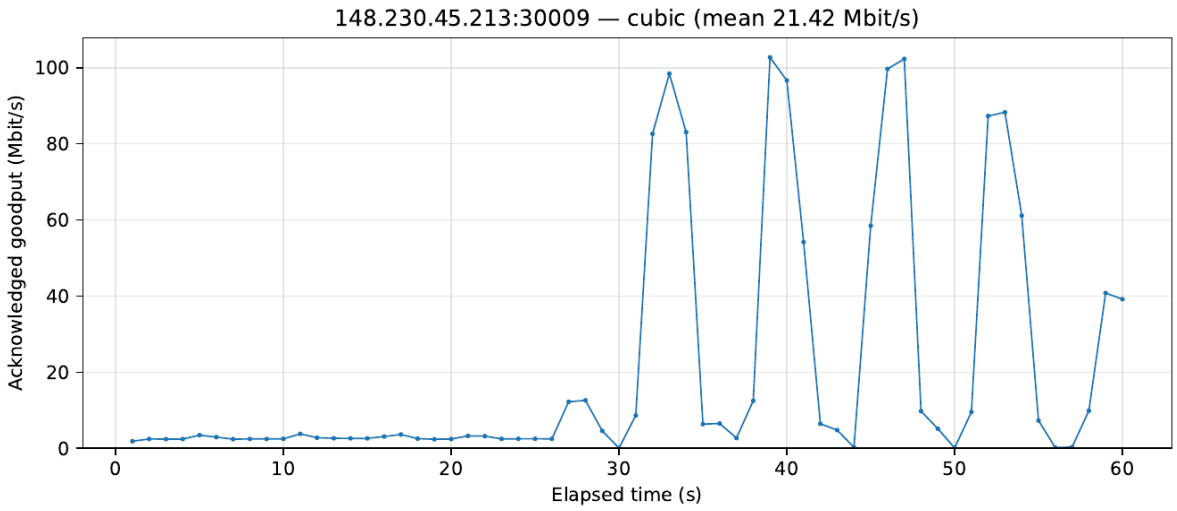


Figure 6: Goodput vs. Time - 148.230.45.213

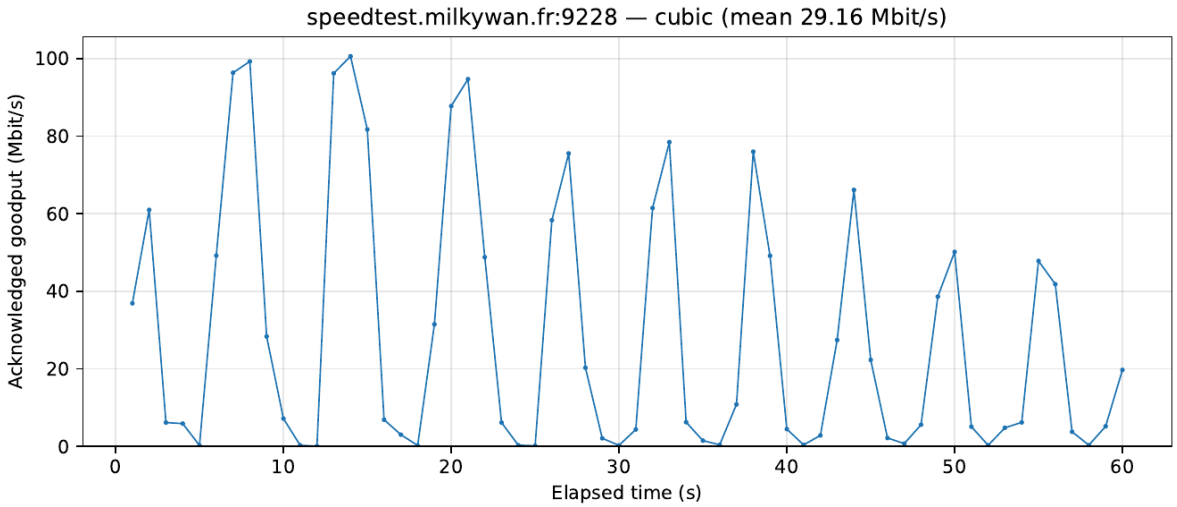


Figure 7: Goodput vs. Time - speedtest.milkywan.fr

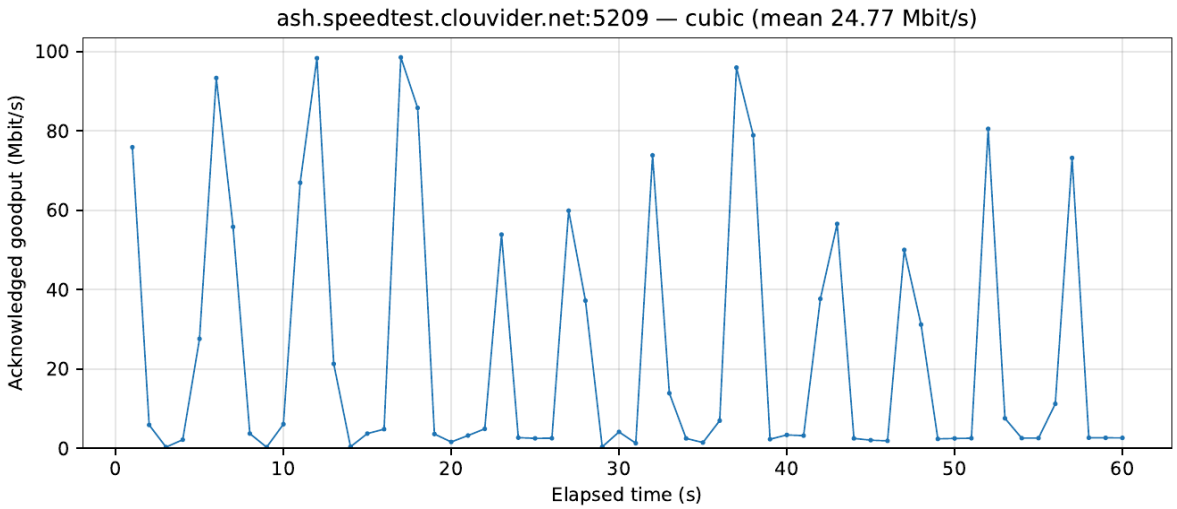


Figure 8: Goodput vs. Time - ash.speedtest.clouvider.net

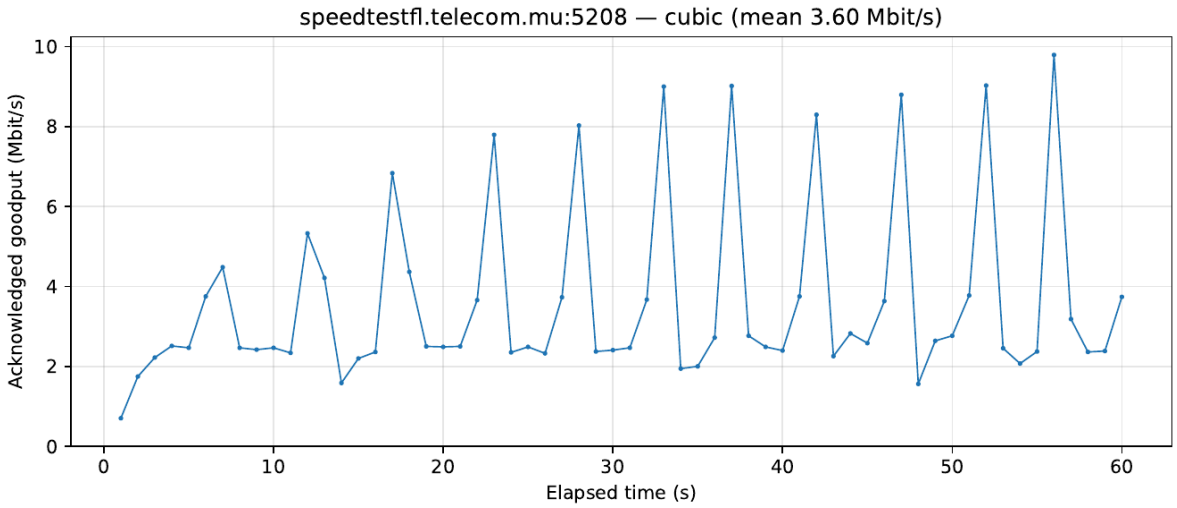


Figure 9: Goodput vs. Time - speedtestfl.telecom.mu

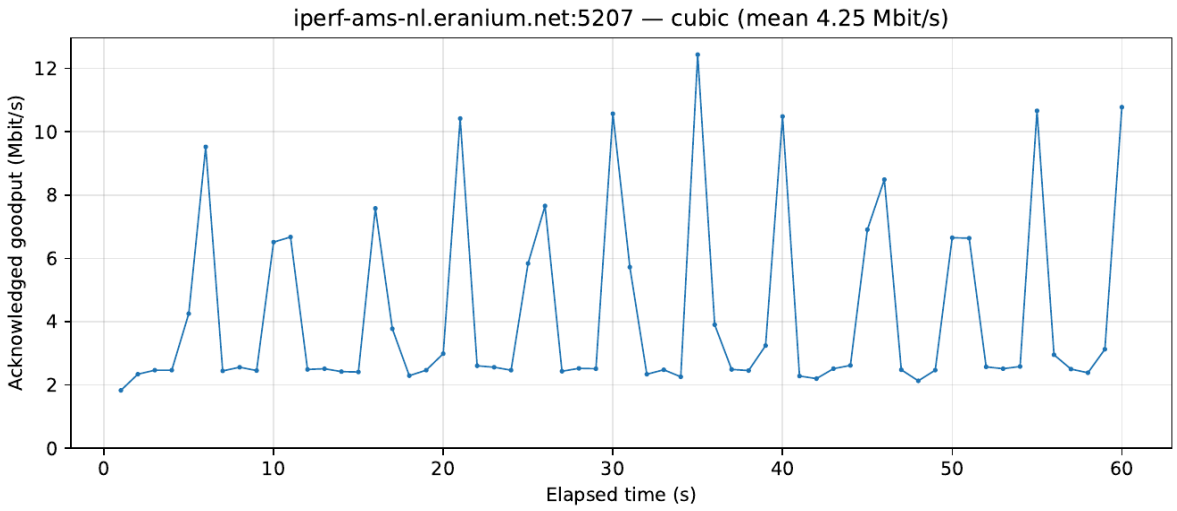


Figure 10: Goodput vs. Time - iperf-ams-nl.eranium.net

Table 1: Goodput Summary for Each Destination (CUBIC)

run_id	host	port	provider	country	algorithm	samples	bytes_sent	min_mbps	median_mbps	mean_mbps	p95_mbps
481d4a1086c3	speedtest.mia11.us.leaseweb.net	5202	LeaseWeb	US	cubic	60	38141952	1.26234677	2.39860897	5.00213464	14.645588 63
c227452dd354	speedsrv.multinet24.pl	5307	Multinet24	PL	cubic	60	30146560	1.51720562	2.46184111 9	3.89409267	10.29627213

66054ec79fe2	iperf.online.net	5209	Scaleway	FR	cubic	60	31719424	1.41284821	2.53738134	4.11833870	11.4931234
fd0e07a51f0d	ping-ams1.online.net	5203	Scaleway	NL	cubic	60	29360128	1.55214020	2.43861047	3.821364	9.97392135
36ea8996c543	speedtestlondon.telecom.mu	5208	Mauritius Telecom	MU	cubic	60	24117248	0.338647496	2.59322052	3.03819532	5.31680930
4e54e6a7965d	148.230.45.213	30009	FortiSASE	ES	cubic	60	163184640	0.057902258	3.54475685	21.41575597	98.42516144
9d24a5e75eeb	speedtest.milkywan.fr	9228	Milky Wan	FR	cubic	60	221380608	0.081035047	7.03630375	29.16367127	96.186303 2
9dcba08a2065	ash.speedtest.clouvider.net	5209	Clouvider	US	cubic	60	188481536	0.277843122 4	3.92074720 7	24.76831674	93.35084381
900239278de8	speedtestfl.telecom.mu	5208	Mauritius Telecom	MU	cubic	60	28704768	0.706431103 3	2.50282147 5	3.59892681 2	9.00074932
2ee64b3ddbb0	iperf-ams-nl.eranium.net	5207	Eranium	NL	cubic	60	34734080	1.82967394	2.54294221 8	4.25480602 8	10.5672130 4

2a) TCP Statistics Collection

- TCP statistics are collected periodically from the live data socket using Linux's TCP_INFO through getsockopt() (part1/run_test.py lines 185-225, part1/tcp_stats.py lines 78-141). We record the congestion window (cwnd), RTT, retransmissions and goodput, along with additional metrics such as RTT variation and pacing rate (part1/tcp_stats.py lines 122-141).
- We use the increase in retransmissions between samples as our estimate of packet loss (part1/tcp_stats.py lines 209-212).
- For Part 2, we select the successful run whose mean goodput is closest to the median across all successful runs (part2/run_tcp_stats.py lines 209-233). The

selected data is saved in representative_samples.csv, along with information about the measurements, units and TCP_INFO decoding in representative.json (part2/run_tcp_stats.py lines 369-518).

2b) TCP Statistics Visualizations

- The plots are generated by plot_results.py (lines 140-424), which creates tcp_stats.pdf for the representative destination.
- For our run, the representative destination was iperf.online.net:5209, using the CUBIC congestion control algorithm.

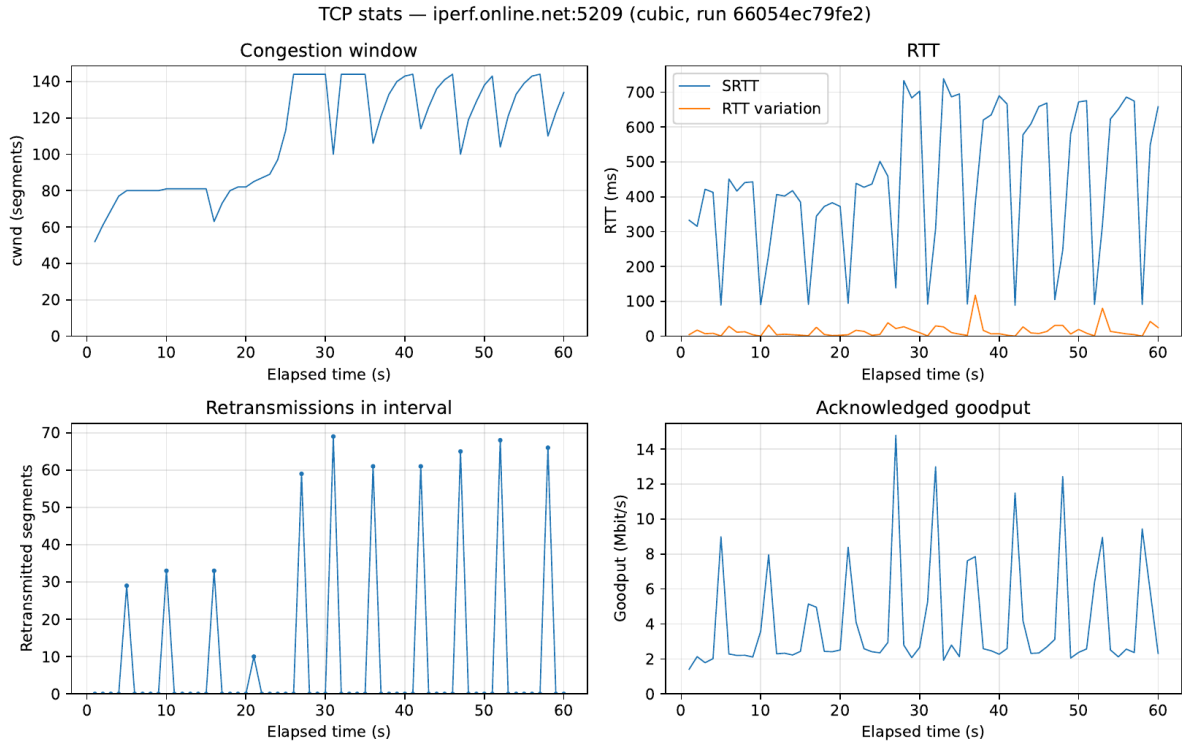


Figure 11: TCP Statistics Over Time (CUBIC)

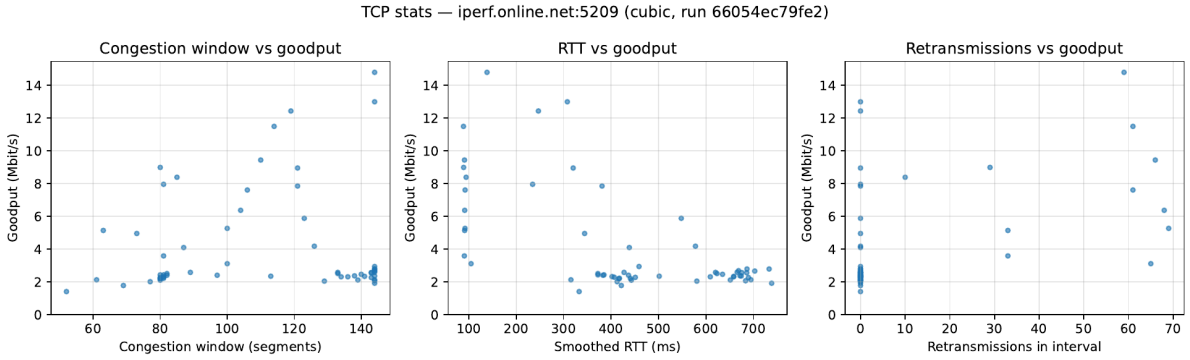


Figure 12: Relationship Between TCP Statistics and Goodput (CUBIC)

2c) Observations

- The congestion window (cwnd) generally increases over time, but sometimes drops. A larger cwnd allows more data to be sent, but it does not always mean higher goodput. This is because other factors, like RTT and network congestion, also affect goodput.
- RTT changes a lot during the experiment, sometimes reaching around 700 ms. The plots show that goodput is usually lower when RTT is high. This could be because packets are taking longer to travel through the network.
- Retransmissions also show several spikes, meaning TCP had to resend some data. This can happen due to packet loss. However, these spikes do not always cause an immediate drop in goodput.

Overall, the plots show that goodput depends on multiple factors, including cwnd, RTT, and retransmissions, rather than just one.

3a) CUBIC vs. Reno vs. BBR

- Running the experiments with all 3 algorithms: `Assignment2/part3/run_algorithms.py`, `run_part3`, lines 224–390. The repetition and rotating-order loop is lines 279–323.
- Setting and verifying the algorithm on the data socket: `Assignment2/part1/run_test.py`, lines 150–170
- Plots: `Assignment2/plot_results.py`, `plot_comparison`, lines 601–678
- Command: `python3 main.py -n 5 -d 60 -i 0.2 -r 3 --seed 42` (Linux 6.8.0-147-generic, aarch64 Ubuntu 24.04 VM)

Figure 13: TCP Statistics Over Time (CUBIC)

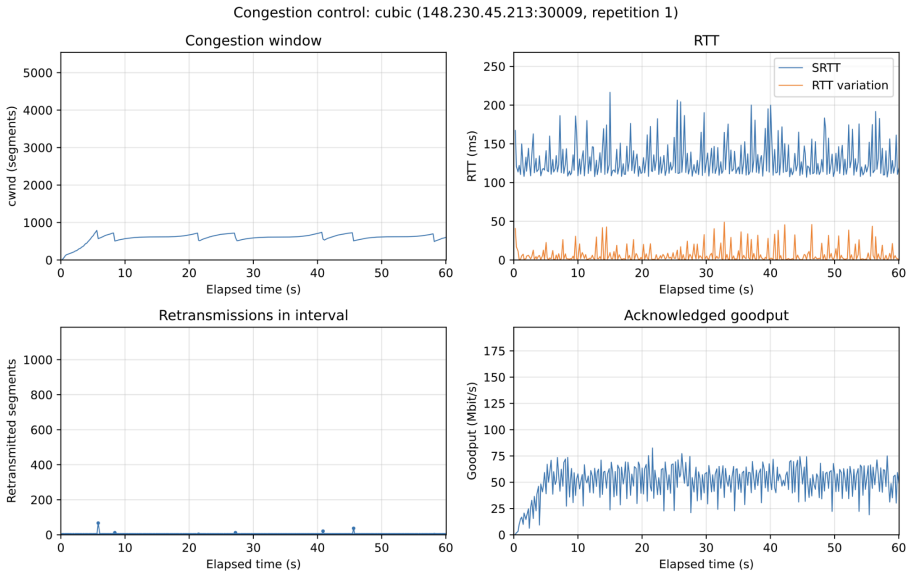


Figure 14: TCP Statistics vs. Goodput (CUBIC)

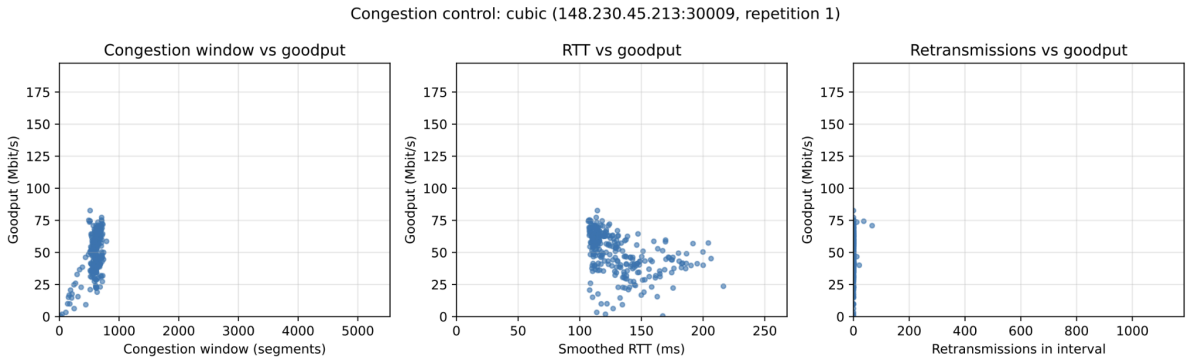


Figure 15: TCP Statistics Over Time (Reno)

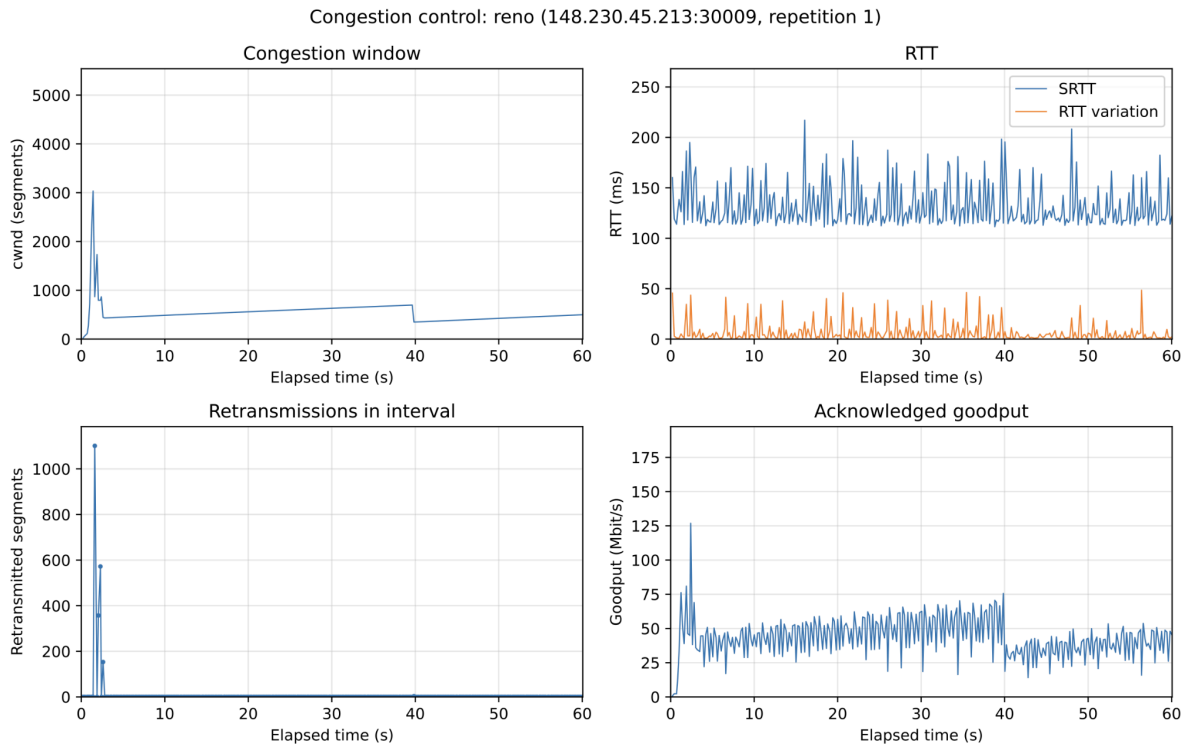


Figure 16: TCP Statistics vs. Goodput (Reno)

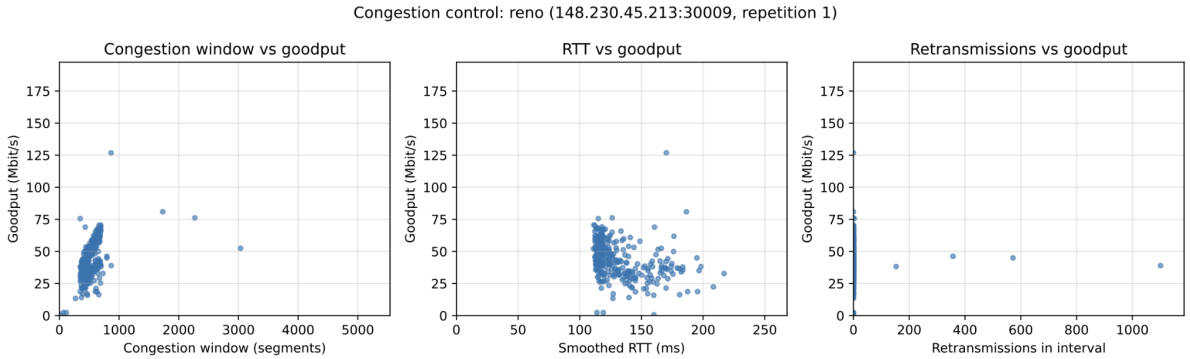


Figure 17: TCP Statistics Over Time (BBR)

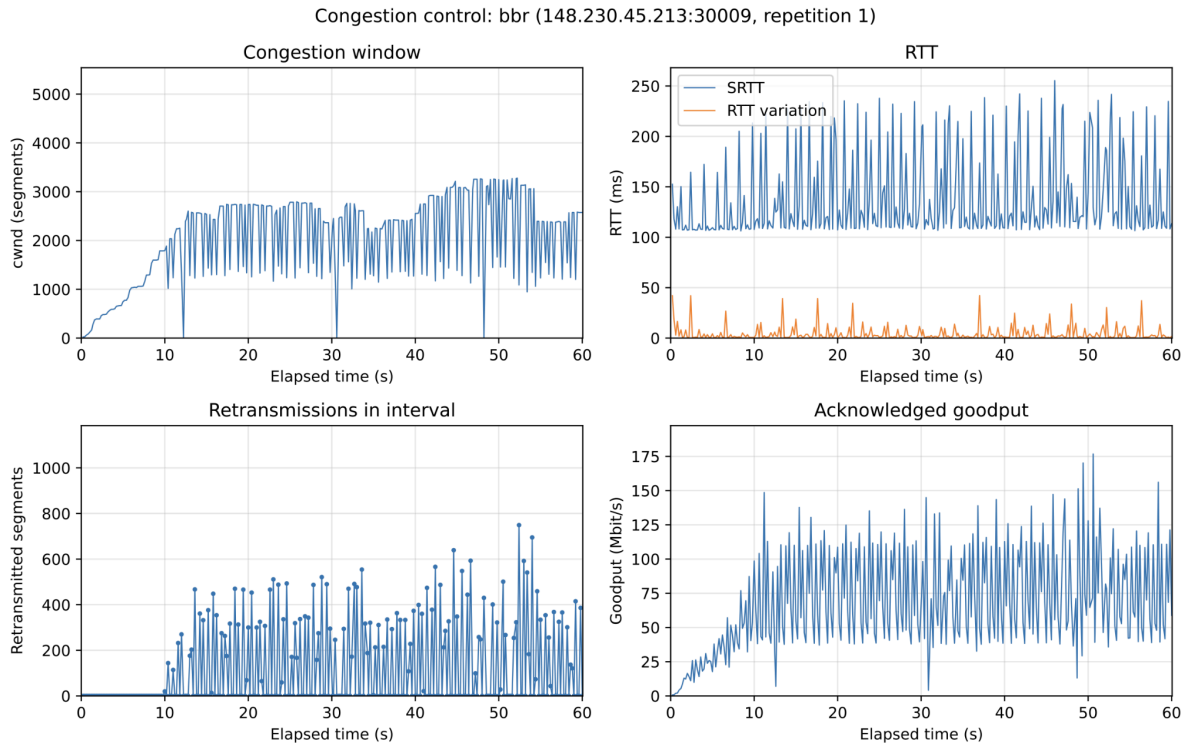


Figure 18: TCP Statistics vs. Goodput (BBR)

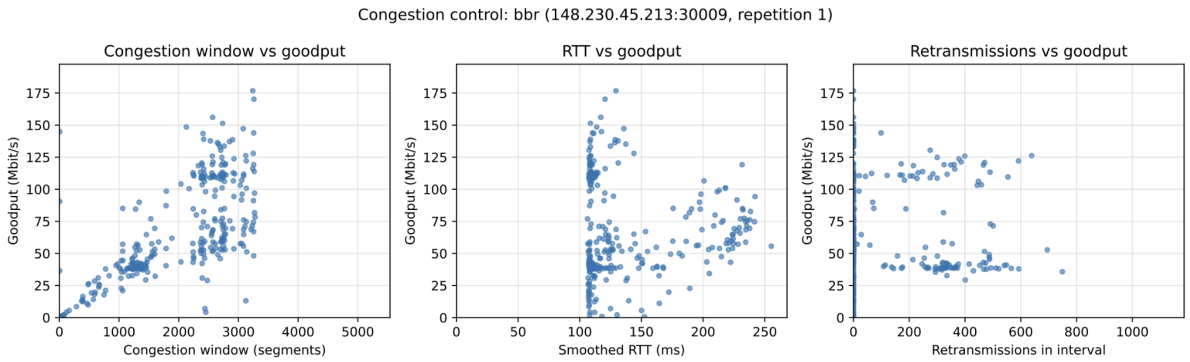


Table 2: Goodput Comparison of CUBIC, Reno and BBR

Destination (min RTT)	Algorithm	Runs	Min (Mbit/s)	Median (Mbit/s)	Mean (Mbit/s)	P95 (Mbit/s)	SD across runs (Mbit/s)	Mean RTT (ms)

LeaseWeb US (48 ms)	CUBIC	3	4.0	96.4	93.2	134.9	1.4	77
	Reno	2	2.9	83.8	85.7	131.0	3.7	72
	BBR	3	0.3	141.8	**144.5**	280.0	11.9	91
FortiSASE ES (105 ms)	CUBIC	3	0.4	51.5	48.6	69.2	1.4	133
	Reno	3	0.0	44.6	45.0	66.8	1.6	129
	BBR	3	0.0	58.2	**67.5**	125.7	1.4	141
MilkyWan FR (109 ms)	CUBIC	3	0.5	50.5	47.1	68.6	2.0	132
	Reno	3	0.5	43.5	45.0	68.0	1.4	132
	BBR	3	0.0	55.7	**72.8**	145.2	6.2	142
Multinet24 PL (130 ms)	CUBIC	3	0.3	43.8	41.5	66.1	3.9	153
	Reno	3	0.4	35.1	36.0	60.9	2.1	155
	BBR	3	0.0	46.2	**63.3**	129.8	5.3	168

Mauritius Telecom (288 ms)	CUBIC	3	0.0	31.1	30.0	60.3	5.5	321
	Reno	3	0.0	20.6	20.2	33.3	2.5	316
	BBR	3	0.0	33.2	**42.4**	116.1	1.9	332

Comparison:

Throughput: BBR had the highest goodput on every destination in every run, 1.5–2.1× Reno's. CUBIC beat Reno in 13 of 14 comparisons, by a small margin on short paths (1.05–1.15×) but 1.48× on the 288 ms path.

RTT: CUBIC and Reno had similar RTT. BBR's RTT was 12–19 ms higher on every destination.

Why:

- **cwnd and loss:** our paths were loss-limited. CUBIC and Reno cut their window on every loss, so they stayed at ~530–610 segments no matter the RTT, and their goodput (≈ cwnd/RTT) dropped on longer paths. BBR ignores loss and sets its rate from estimated bandwidth, so it kept going faster.
- **CUBIC vs. Reno:** CUBIC cuts less on loss (to 70% vs. 50%) and regrows based on time, not RTT. Reno adds one segment per RTT, which is very slow on long paths. That's why CUBIC's lead was biggest at 288 ms.
- **Queueing:** BBR keeps up to 2× the BDP in flight, which overfills router buffers. The result is higher RTT and 7–16% retransmitted bytes, vs. under 1% for CUBIC and Reno.
- **BDP:** CUBIC and Reno's ~0.8 MB windows were close to the BDP on short paths but well below it on the 288 ms path (up to 1.5 MB), where they fell furthest behind. BBR's windows were well above the BDP.

When each is better: BBR for raw throughput on long paths. CUBIC over Reno on long paths, with little difference on short ones. CUBIC or Reno when low delay and few retransmissions matter.

Variability: results on the same server were consistent, usually within 2–9% across runs and up to 18% on the longest path. Different servers varied 3–4× in goodput. The ranking held in every repetition. Public Internet tests aren't fully consistent: traffic, routes and server load change over time, servers can be busy (one test was refused 3 times), and our VM on Wi-Fi adds some jitter.

Runs and order: 3 repetitions × 5 destinations × 3 algorithms = 45 tests, 60 s each, sampled every 0.2 s. Each destination ran all three algorithms back to back, and the order rotated each repetition: CUBIC → Reno → BBR, then Reno → BBR → CUBIC, then BBR → CUBIC → Reno. 44 of 45 tests completed; one Reno test was refused by a busy server.

3b) Our Congestion Control Algorithm

Idea and evidence

Our algorithm, RTT–Loss AIMD (RLA), grows the congestion window when the connection is using it well, backs off when RTT rises, and cuts back more strongly when TCP detects loss. The goal is to get good throughput without building an unnecessarily large queue.

Our measurements suggest that one signal alone is not enough. In a five-destination comparison with 60-second transfers, BBR at LeaseWeb averaged 144.5 Mbit/s versus CUBIC's 93.2 Mbit/s, but its mean RTT was 90.9 ms versus 76.5 ms and its retransmitted-byte fraction was 12.64% versus 0.04%. On Mauritius, CUBIC averaged 30.0 Mbit/s versus Reno's 20.2 Mbit/s despite similar mean RTTs (321 and 316 ms). In a separate four-destination comparison with 20-second transfers, the ranking changed: at Multinet24, Reno averaged 151.8 Mbit/s versus CUBIC's 98.1 Mbit/s and BBR's 147.0 Mbit/s, with no retransmitted segments in those Part 3 tests. These observations motivate using RTT to detect increasing delay, loss feedback to trigger stronger backoff, and acknowledged throughput to check whether the current window is being used effectively. They motivate RLA's design but do not establish its performance.

Compared with Reno, CUBIC, and BBR

Reno grows its window roughly one segment per RTT in congestion avoidance and halves it on loss. It is simple, but recovery can be slow on high-capacity, long-RTT paths. RLA keeps the simple additive/loss-backoff idea and adds delay and throughput checks; those checks can also make it grow slower.

CUBIC uses a time-based cubic growth curve to probe capacity faster on large-window paths. Its loss-based probing can build queues. RLA tries to back off before loss when delay rises, trading some throughput for less queueing.

BBR estimates bandwidth and minimum RTT to control its sending rate. It can achieve high throughput without waiting for loss, but depends on the quality of those estimates and can have sharing issues with competing flows. RLA is simpler and uses throughput only to decide whether to grow; it may leave more bandwidth unused.

How the window changes

Starting up. Keep Linux's initial window, normally ten segments, and use standard slow start: increase the window as packets are acknowledged. This avoids spending the transfer at a tiny window.

Regular updates. Use a control interval of at least 100 ms and at least the RTT measured at the interval's start. Remember the lowest positive smoothed RTT seen so far as the baseline. Measure goodput as eight times newly acknowledged bytes divided by the actual elapsed time. Compare it with the starting window's approximate supported rate:

$$\text{window rate} = \frac{8 \times \text{cwnd} \times \text{MSS}}{\text{RTT}} \quad \text{bits/s, with RTT in seconds.}$$

Here, cwnd is in segments and MSS is bytes per segment. This rate is only an approximation, not a measurement of the link's capacity.

Backing off. At a completed control interval, if RTT rises more than 25% above the baseline, reduce cwnd by 10% and leave slow start. On a native TCP loss/congestion event, request a threshold of half the window and let Linux handle recovery. Respond once through that recovery mechanism, rather than halving for every retransmitted packet in a burst.

Growing carefully. After slow start, add one segment per completed interval only if RTT is less than 10% above baseline, goodput is at least 80% of the window rate, and the sender is constrained by its congestion window. Otherwise hold it steady. The gap between the RTT thresholds avoids switching between growth and backoff for every small RTT change. The throughput check avoids enlarging a window that is not being fully used.

Terms

Term	Meaning
ACK	An acknowledgment telling the sender that data arrived.
cwnd	Congestion window: the limit on data sent but not yet acknowledged, measured here in segments.
MSS	Maximum segment size: the number of data bytes in a full TCP segment.
RTT	Round-trip time: how long data and its acknowledgment take to travel. We use a smoothed estimate.
Baseline RTT	The lowest positive smoothed RTT seen so far; a reference for spotting extra delay.
Goodput	The rate of newly acknowledged data, counting each byte once even if retransmitted.
Queueing	Packets waiting in network buffers, which adds delay and can raise RTT.
Slow start	The initial phase where ACKs let the window grow quickly.
AIMD	Additive increase, multiplicative decrease: grow by a fixed amount and back off by a fraction.

Pseudocode

```
Start with Linux's initial cwnd and slow-start threshold.
Remember the lowest positive smoothed RTT as base_RTT.
At each interval start, save time, ACKed bytes, cwnd, MSS, RTT.
Interval length = max(100 ms, starting RTT).

On a TCP loss/congestion event:
    threshold = max(2, floor(cwnd / 2))
    let Linux handle recovery; discard the current interval

On an ACK outside recovery:
    skip invalid measurements
    update base_RTT with the positive smoothed RTT
    if no interval is active, begin one
    if interval finished:
        goodput = 8 * ACKed bytes since interval start / elapsed time
        window_rate = 8 * starting cwnd * starting MSS / starting RTT
    if interval finished and RTT > 1.25 * base_RTT:
        cwnd = max(2, floor(0.90 * cwnd))
        threshold = cwnd
    else if in slow start and constrained by cwnd:
        cwnd += newly ACKed segments, up to the threshold
    else if interval finished and RTT < 1.10 * base_RTT:
        if goodput >= 0.80 * window_rate and constrained by cwnd:
            cwnd += 1
    otherwise hold cwnd

    respect Linux's maximum window
    if interval finished, begin a new interval
```

Skip invalid measurements and restart the interval after recovery or an idle restart. RTT is expressed in seconds in the rate equations. Two segments is the normal minimum, but Linux can temporarily use one after a timeout. The 10%/25% RTT thresholds provide a hold region, the 10% delay backoff is gentler than the loss response, and the 80% utilization check discourages growth when the old window is underused. These are heuristic choices, not values optimized or validated by our experiments.

Fairness, stability, and limitations

Fairness. With similar RTTs and packet sizes, flows eligible for the same additive increases and experiencing congestion together should tend toward equal windows: equal additive increments preserve the window difference, while a shared percentage decrease shrinks it. In practice, longer-RTT flows update less often, low-goodput flows may fail the growth check, and RLA may yield to CUBIC or Reno by backing off earlier. Fairness is not guaranteed.

Stability. Growth at low delay and backoff at high delay should keep the rate near an operating range on a stable path. Holding between the two thresholds should reduce small fluctuations. We expect some oscillation, rather than exact convergence, especially with delayed feedback or changing traffic. The utilization condition can also prevent further growth at an underused operating point, so convergence to full utilization is not guaranteed.

Limitations. We assume usable measurements and a reasonably stable path. Route changes can make the minimum RTT outdated, wireless loss can cause unnecessary backoff, and ACK bursts can distort throughput. A minimum smoothed RTT can already contain queueing delay. The growth check can leave spare bandwidth unused. Our measurements have not tested competing RLA flows or established fairness and stability.

Reproducibility / Docker

main.py, lines 105–142 runs the complete pipeline: destination selection and goodput tests, representative TCP-statistics plots, repeated CUBIC/Reno/BBR tests, and PDF generation. Options (main.py, lines 49–96) include -n destinations, -d seconds per transfer, -i sampling seconds, -r comparison repetitions, --algorithm for Parts 1/2, --seed, --servers, and --output. The default input is `data/listed_iperf3_servers.json`.

The Dockerfile uses FROM ubuntu:24.04, Python 3, and matplotlib (from requirements.txt). Run on Linux or Windows/WSL2 with CUBIC, Reno, and BBR available and permitted. Docker uses the host kernel, so record its version and configuration:

- uname -r
- sysctl net.ipv4.tcp_available_congestion_control
- sysctl net.ipv4.tcp_allowed_congestion_control
- sysctl net.ipv4.tcp_congestion_control

If needed, load BBR on the host with sudo modprobe tcp_bbr. From the Assignment2/ folder, build and run the full pipeline from docker:

- ./scripts/docker.sh build
- ./scripts/docker.sh run -n 10 -d 60 -i 0.2 -r 3 --algorithm cubic

The scripts/docker.sh, lines 13–18 uses host networking and persists output/. Outputs include raw CSV/JSON measurements, summaries, failures, manifests, and goodput.pdf, tcp_stats.pdf, and comparison.pdf.

The command above is the recommended full run. Our separate Docker rerun used:

TESTED ON: Linux 6.18.50, x86_64.

AI Usage

We used AI tools (ChatGPT, Codex + GPT-6.1 Sol, and qwen3.8:27b, qwen3.8:4b) to help across all parts of the assignment. Core use cases (beyond helping general development) were: finding bugs in our iperf client implementation, reviewing/analyzing run output, and helping set up the Docker system. It was also used to provide a final pass on the report (looking for consistency and formatting, checking the report against the handout and source).